UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2021/2022

MAY EXAMINATION

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

TUESDY, 24 MAY 2022 TIME : 9.00 AM – 11.0 AM (2 HOURS)

BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)
COMMUNICATIONS AND NETWORKING
BACHELOR OF COMPUTER SCIENCE (HONOURS)
BACHELOR OF INFORMATION SYSTEMS (HONOURS)
INFORMATION SYSTEMS ENGINEERING

Instruction to Candidates:

This question paper consists of **TWO (2)** sections.

SECTION A consists of **THREE (3)** questions. **Answer ALL questions.**

SECTION B consists of **TWO (2)** questions. Answer **ONLY ONE (1)** question. Should a candidate answer more than ONE (1) questions in SECTION B, marks will only be awarded for the FIRST question in the order the candidate submits the answers.

Each question carries 25 marks.

Candidates are allowed to use a calculator.

Answer questions only in the answer booklet provided.

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

SECTION A (Answer ALL questions)

Q1. (a) Answer the following questions. Show all your workings.

(i) Convert 495_7 to decimal and BCD. (4 marks)

(ii) Solve the arithmetic operation $1010\ 1101_{Gray} - 128_{12}$ in binary. (6 marks)

(iii) Simplify the function $f = \overline{x + \bar{x}.y + \bar{x}.\bar{y}} + \overline{x + \bar{y}}$ as much as possible using Boolean algebra techniques. (5 marks)

(b) Figure 1.1 shows a three inputs combinational circuit. Answer the following question by referring to this figure.

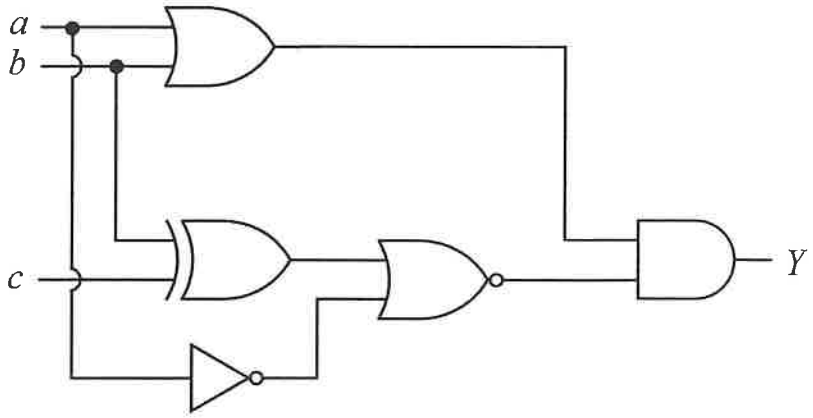


Figure 1.1

(i) Derive the Boolean expression and determine its minterm list. (6 marks)

(ii) Construct the corresponding truth table for the Boolean expression in Q1(b)(i). (4 marks)

[Total : 25 marks]

Q2. (a) Figure 2.1 shows a MIPS code segment being stored in the memory starting at memory location 0x004002BC.

```
label:   lw   $t2, 0($s0)
         addi $s0, $s0, 4
         andi $t3, $t2, 0xF0F0
         blt  $t2, $t3, label
```

Figure 2.1

(i) Given the initial content of register `$s0` is `0x2000AB3F`. Determine its new value after the `addi` instruction. (2 marks)

(ii) Given the content of register `$t2` is `0xFCCD1133`, determine the value of `$t3` after the instruction `andi $t3, $t2, 0xF0F0`. Show the workings. Give the final answer in hexadecimal. (4 marks)

(iii) What are the two possible values for the Program Counter (PC) after the branch instruction has executed? (4 marks)

(iv) Identify a pseudoinstruction from the code above. State the operation of this pseudoinstruction. (3 marks)

(b) (i) Elaborate how we can identify the instruction format (i.e., R-, I- or J-type) of a machine code. (5 marks)

(ii) Translate the following machine code to MIPS assembly code. Show all your workings.

`0x0125882A`

(7 marks)
[Total : 25 marks]

Q3. (a) In a MIPS based processor's memory hierarchy system, memory allocation is divided into stack, data and text segment. Answer the following.

(i) Describe the TWO (2) types of data segments and indicate the object that is held in each of these segments. (8 marks)

(ii) Explain the purposes of using the stack segment. (3 marks)

(b) In the processor's internal data communication medium, bus width is a critical parameter in determining the whole system performance.

(i) Elaborate the significance of data bus width. (3 marks)

(ii) Elaborate the significance of address bus width. (3 marks)

(c) Describe TWO (2) of the data transfer modes between I/O peripherals and main memory. (8 marks)

[Total : 25 marks]

SECTION B (Choose only ONE question)

Q4. (a) Figure 4.1 shows a fragment of a MIPS assembly code. Answer the following questions by referring to this figure.

```
Address              Assembly code
  ...                    ...
0x004000A4  Loop: and  $t2, $t1, $t9
0x004000A8        srl  $t2, $t2, $t7
0x004000AC        li   $v0, 1
0x004000B0        move $a0, $t2
0x004000B4        syscall
0x004000B8        srl       $t9, $t9, 3
0x004000BC        addi $t8, $t8, -1
0x004000C0        addi $t7, $t7, -3      # Instruction X
0x004000C0        bne  $t8, $zero, Loop
0x004000C0        jr   $ra               # Instruction Y
0x004000C0  Exit:
```

Figure 4.1.

(i) Name the addressing mode of Instruction X in Figure 4.1. (2 marks)

(ii) Describe the addressing mode of Instruction X with the help of a diagram. (6 marks)

(iii) Compare the addressing mode for Instruction X and Instruction Y by providing TWO (2) similarities and TWO (2) differences. (6 marks)

(b) A designer noticed that the single cycle processor has a slower clock rate compared to a multicycle processor. To improve the performance, would you advise the designer to increase the clock rate of the single cycle to be as fast as the multicycle proessor? Why? Explain your answer by discussing the differences between single cycle and multi-cycle micrcoarchitectures.

(11 marks)

[Total : 25 marks]

Q5. (a) With the aid of a suitable diagram, briefly explain the steps required to translate a program from the high-level language to program execution stage. (9 marks)

(b) Compilers may have a profound impact on the program performance. Table 5.1 shows the number of instruction and execution time of running the same program when two different compilers are used.

	No. of instruction	Execution time
Compiler A	1.0×10^9	1.8 s
Compiler B	1.2×10^9	1.5 s

Table 5.1

(i) Given that the processor has a clock cycle time of 1ns, calculate the average clock cycles per instruction (CPI) for each program. Show all your workings. (4 marks)

(ii) A new compiler (Compiler C) is developed that uses only 9.0×10^8 instructions and has an average CPI of 1.5. Which compiler, among the three, has the highest performance? Justify your answer. (4 marks)

(c) Explain, from the four design principles perspective, why RISC ISA based system's performance not necessarily inferior than CISC despite the fact it is compiled to more instructions for the same application code. (8 marks)

[Total : 25 marks]

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

Appendix A: MIPS Reference Data

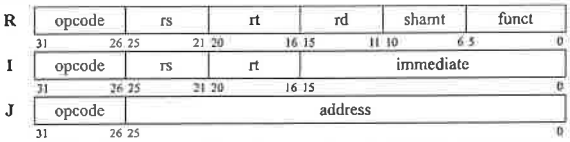
MIPS Reference Data ①

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	0 / 20_{hex}
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		0 / 21_{hex}
And	and	R	R[rd] = R[rs] & R[rt]		0 / 24_{hex}
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}
Branch On Equal	beq	I	if(R[rs]==R[rt]) PC=PC+4+BranchAddr	(4)	4_{hex}
Branch On Not Equal	bne	I	if(R[rs]!=R[rt]) PC=PC+4+BranchAddr	(4)	5_{hex}
Jump	j	J	PC=JumpAddr	(5)	2_{hex}
Jump And Link	jal	J	R[31]=PC+8;PC=JumpAddr	(5)	3_{hex}
Jump Register	jr	R	PC=R[rs]		0 / 08_{hex}
Load Byte Unsigned	lbu	I	R[rt]={24'b0,M[R[rs]+SignExtImm](7:0)}	(2)	24_{hex}
Load Halfword Unsigned	lhu	I	R[rt]={16'b0,M[R[rs]+SignExtImm](15:0)}	(2)	25_{hex}
Load Linked	ll	I	R[rt] = M[R[rs]+SignExtImm]	(2,7)	30_{hex}
Load Upper Imm.	lui	I	R[rt] = {imm, 16'b0}		f_{hex}
Load Word	lw	I	R[rt] = M[R[rs]+SignExtImm]	(2)	23_{hex}
Nor	nor	R	R[rd] = ~ (R[rs] | R[rt])		0 / 27_{hex}
Or	or	R	R[rd] = R[rs] | R[rt]		0 / 25_{hex}
Or Immediate	ori	I	R[rt] = R[rs] | ZeroExtImm	(3)	d_{hex}
Set Less Than	slt	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0		0 / $2a_{hex}$
Set Less Than Imm.	slti	I	R[rt] = (R[rs] < SignExtImm)? 1 : 0	(2)	a_{hex}
Set Less Than Imm. Unsigned	sltiu	I	R[rt] = (R[rs] < SignExtImm) ? 1 : 0	(2,6)	b_{hex}
Set Less Than Unsig.	sltu	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0	(6)	0 / $2b_{hex}$
Shift Left Logical	sll	R	R[rd] = R[rt] << shamt		0 / 00_{hex}
Shift Right Logical	srl	R	R[rd] = R[rt] >> shamt		0 / 02_{hex}
Store Byte	sb	I	M[R[rs]+SignExtImm](7:0) = R[rt](7:0)	(2)	28_{hex}
Store Conditional	sc	I	M[R[rs]+SignExtImm] = R[rt]; R[rt] = (*atomic*) ? 1 : 0	(2,7)	38_{hex}
Store Halfword	sh	I	M[R[rs]+SignExtImm](15:0) = R[rt](15:0)	(2)	29_{hex}
Store Word	sw	I	M[R[rs]+SignExtImm] = R[rt]	(2)	$2b_{hex}$
Subtract	sub	R	R[rd] = R[rs] - R[rt]	(1)	0 / 22_{hex}
Subtract Unsigned	subu	R	R[rd] = R[rs] - R[rt]		0 / 23_{hex}

(1) May cause overflow exception
(2) SignExtImm = { 16{immediate[15]}, immediate }
(3) ZeroExtImm = { 16{1b'0}, immediate }
(4) BranchAddr = { 14{immediate[15]}, immediate, 2'b0 }
(5) JumpAddr = { PC+4[31:28], address, 2'b0 }
(6) Operands considered unsigned numbers (vs. 2's comp.)
(7) Atomic test&set pair; R[rt] = 1 if pair atomic, 0 if not atomic

BASIC INSTRUCTION FORMATS



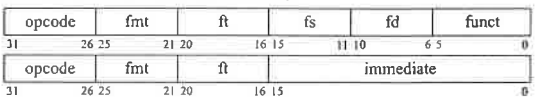
ARITHMETIC CORE INSTRUCTION SET ②

NAME, MNEMONIC		FOR-MAT	OPERATION		OPCODE / FMT /FT / FUNCT (Hex)
Branch On FP True	bc1t	FI	if(FPcond)PC=PC+4+BranchAddr	(4)	11/8/1/--
Branch On FP False	bc1f	FI	if(!FPcond)PC=PC+4+BranchAddr	(4)	11/8/0/--
Divide	div	R	Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]		0/--/--/1a
Divide Unsigned	divu	R	Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]	(6)	0/--/--/1b
FP Add Single	add.s	FR	F[fd]= F[fs] + F[ft]		11/10/--/0
FP Add Double	add.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} + {F[ft],F[ft+1]}		11/11/--/0
FP Compare Single	c.x.s*	FR	FPcond = (F[fs] *op* F[ft]) ? 1 : 0		11/10/--/*y*
FP Compare Double	c.x.d*	FR	FPcond = ({F[fs],F[fs+1]} *op* {F[ft],F[ft+1]}) ? 1 : 0		11/11/--/*y*

* (*x* is eq, lt, or le) (*op* is ==, <, or <=) (*y* is 32, 3c, or 3e)

FP Divide Single	div.s	FR	F[fd] = F[fs] / F[ft]		11/10/--/3
FP Divide Double	div.d	FR	{F[fd].F[fd+1]} = {F[fs],F[fs+1]} / {F[ft],F[ft+1]}		11/11/--/3
FP Multiply Single	mul.s	FR	F[fd] = F[fs] * F[ft]		11/10/--/2
FP Multiply Double	mul.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} * {F[ft],F[ft+1]}		11/11/--/2
FP Subtract Single	sub.s	FR	F[fd]=F[fs] - F[ft]		11/10/--/1
FP Subtract Double	sub.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} - {F[ft],F[ft+1]}		11/11/--/1
Load FP Single	lwc1	I	F[rt]=M[R[rs]+SignExtImm]	(2)	31/--/--/--
Load FP Double	ldc1	I	F[rt]=M[R[rs]+SignExtImm]; F[rt+1]=M[R[rs]+SignExtImm+4]	(2)	35/--/--/--
Move From Hi	mfhi	R	R[rd] = Hi		0 /--/--/10
Move From Lo	mflo	R	R[rd] = Lo		0 /--/--/12
Move From Control	mfc0	R	R[rd] = CR[rs]		10 /0/--/0
Multiply	mult	R	{Hi,Lo} = R[rs] * R[rt]		0/--/--/18
Multiply Unsigned	multu	R	{Hi,Lo} = R[rs] * R[rt]	(6)	0/--/--/19
Shift Right Arith.	sra	R	R[rd] = R[rt] >>> shamt		0/--/--/3
Store FP Single	swc1	I	M[R[rs]+SignExtImm] = F[rt]	(2)	39/--/--/--
Store FP Double	sdc1	I	M[R[rs]+SignExtImm] = F[rt]; M[R[rs]+SignExtImm+4] = F[rt+1]	(2)	3d/--/--/--

FLOATING-POINT INSTRUCTION FORMATS



PSEUDOINSTRUCTION SET

NAME	MNEMONIC	OPERATION
Branch Less Than	blt	if(R[rs]<R[rt]) PC = Label
Branch Greater Than	bgt	if(R[rs]>R[rt]) PC = Label
Branch Less Than or Equal	ble	if(R[rs]<=R[rt]) PC = Label
Branch Greater Than or Equal	bge	if(R[rs]>=R[rt]) PC = Label
Load Immediate	li	R[rd] = immediate
Move	move	R[rd] = R[rs]

REGISTER NAME, NUMBER, USE, CALL CONVENTION

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
$zero	0	The Constant Value 0	N.A.
$at	1	Assembler Temporary	No
$v0-$v1	2-3	Values for Function Results and Expression Evaluation	No
$a0-$a3	4-7	Arguments	No
$t0-$t7	8-15	Temporaries	No
$s0-$s7	16-23	Saved Temporaries	Yes
$t8-$t9	24-25	Temporaries	No
$k0-$k1	26-27	Reserved for OS Kernel	No
$gp	28	Global Pointer	Yes
$sp	29	Stack Pointer	Yes
$fp	30	Frame Pointer	Yes
$ra	31	Return Address	Yes

This question paper consists of 5 questions on 9 printed pages.

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

③

OPCODES, BASE CONVERSION, ASCII SYMBOLS

MIPS opcode (31:26)	(1) MIPS funct (5:0)	(2) MIPS funct (5:0)	Binary	Decimal	Hexadecimal	ASCII Character	Decimal	Hexadecimal	ASCII Character
(1)	sll	add.f	00 0000	0	0	NUL	64	40	@
		sub.f	00 0001	1	1	SOH	65	41	A
j	srl	mul.f	00 0010	2	2	STX	66	42	B
jal	sra	div.f	00 0011	3	3	ETX	67	43	C
beq	sllv	sqrt.f	00 0100	4	4	EOT	68	44	D
bne		abs.f	00 0101	5	5	ENQ	69	45	E
blez	srlv	mov.f	00 0110	6	6	ACK	70	46	F
bgtz	srav	neg.f	00 0111	7	7	BEL	71	47	G
addi	jr		00 1000	8	8	BS	72	48	H
addiu	jalr		00 1001	9	9	HT	73	49	I
slti	movz		00 1010	10	a	LF	74	4a	J
sltiu	movn		00 1011	11	b	VT	75	4b	K
andi	syscall	round.w.f	00 1100	12	c	FF	76	4c	L
ori	break	trunc.w.f	00 1101	13	d	CR	77	4d	M
xori		ceil.w.f	00 1110	14	e	SO	78	4e	N
lui	sync	floor.w.f	00 1111	15	f	SI	79	4f	O
	mfhi		01 0000	16	10	DLE	80	50	P
(2)	mthi		01 0001	17	11	DC1	81	51	Q
	mflo	movz.f	01 0010	18	12	DC2	82	52	R
	mtlo	movn.f	01 0011	19	13	DC3	83	53	S
			01 0100	20	14	DC4	84	54	T
			01 0101	21	15	NAK	85	55	U
			01 0110	22	16	SYN	86	56	V
			01 0111	23	17	ETB	87	57	W
	mult		01 1000	24	18	CAN	88	58	X
	multu		01 1001	25	19	EM	89	59	Y
	div		01 1010	26	1a	SUB	90	5a	Z
	divu		01 1011	27	1b	ESC	91	5b	[
			01 1100	28	1c	FS	92	5c	\
			01 1101	29	1d	GS	93	5d	]
			01 1110	30	1e	RS	94	5e	^
			01 1111	31	1f	US	95	5f	_
lb	add	cvt.s.f	10 0000	32	20	Space	96	60	`
lh	addu	cvt.d.f	10 0001	33	21	!	97	61	a
lwl	sub		10 0010	34	22	"	98	62	b
lw	subu		10 0011	35	23	#	99	63	c
lbu	and	cvt.w.f	10 0100	36	24	$	100	64	d
lhu	or		10 0101	37	25	%	101	65	e
lwr	xor		10 0110	38	26	&	102	66	f
	nor		10 0111	39	27	'	103	67	g
sb			10 1000	40	28	(	104	68	h
sh			10 1001	41	29	)	105	69	i
swl	slt		10 1010	42	2a	*	106	6a	j
sw	sltu		10 1011	43	2b	+	107	6b	k
			10 1100	44	2c	,	108	6c	l
			10 1101	45	2d	-	109	6d	m
swr			10 1110	46	2e	.	110	6e	n
cache			10 1111	47	2f	/	111	6f	o
ll	tge	c.f.f	11 0000	48	30	0	112	70	p
lwc1	tgeu	c.un.f	11 0001	49	31	1	113	71	q
lwc2	tlt	c.eq.f	11 0010	50	32	2	114	72	r
pref	tltu	c.ueq.f	11 0011	51	33	3	115	73	s
	teq	c.olt.f	11 0100	52	34	4	116	74	t
ldc1		c.ult.f	11 0101	53	35	5	117	75	u
ldc2	tne	c.ole.f	11 0110	54	36	6	118	76	v
		c.ule.f	11 0111	55	37	7	119	77	w
sc		c.sf.f	11 1000	56	38	8	120	78	x
swc1		c.ngle.f	11 1001	57	39	9	121	79	y
swc2		c.seq.f	11 1010	58	3a	:	122	7a	z
		c.ngl.f	11 1011	59	3b	;	123	7b	{
		c.lt.f	11 1100	60	3c	<	124	7c	|
sdc1		c.nge.f	11 1101	61	3d	=	125	7d	}
sdc2		c.le.f	11 1110	62	3e	>	126	7e	~
		c.ngt.f	11 1111	63	3f	?	127	7f	DEL

(1) opcode(31:26) = 0

(2) opcode(31:26) = 17_{ten} (11_{hex}); if fmt(25:21)=16_{ten} (10_{hex}) f = s (single); if fmt(25:21)=17_{ten} (11_{hex}) f = d (double)

IEEE 754 FLOATING-POINT STANDARD

$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$

where Single Precision Bias = 127, Double Precision Bias = 1023.

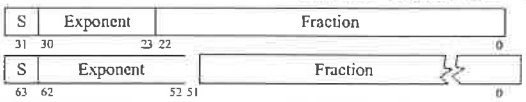
④

IEEE 754 Symbols

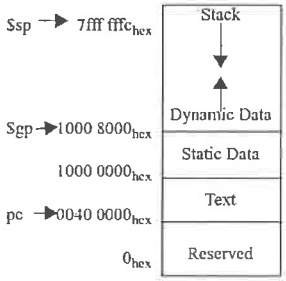
Exponent	Fraction	Object
0	0	± 0
0	≠0	± Denorm
1 to MAX - 1	anything	± Fl. Pt. Num.
MAX	0	±∞
MAX	≠0	NaN

S.P. MAX = 255, D.P. MAX = 2047

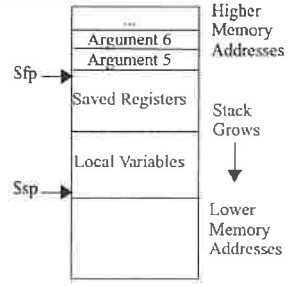
IEEE Single Precision and Double Precision Formats:



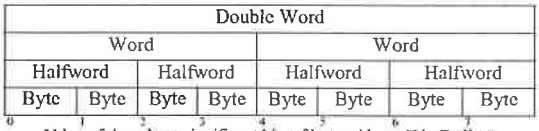
MEMORY ALLOCATION



STACK FRAME

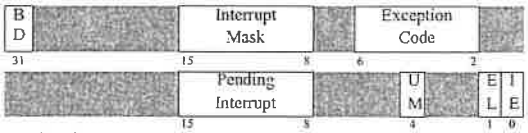


DATA ALIGNMENT



Value of three least significant bits of byte address (Big Endian)

EXCEPTION CONTROL REGISTERS: CAUSE AND STATUS



BD = Branch Delay, UM = User Mode, EL = Exception Level, IE = Interrupt Enable

EXCEPTION CODES

Number	Name	Cause of Exception	Number	Name	Cause of Exception
0	Int	Interrupt (hardware)	9	Bp	Breakpoint Exception
4	AdEL	Address Error Exception (load or instruction fetch)	10	RI	Reserved Instruction Exception
5	AdES	Address Error Exception (store)	11	CpU	Coprocessor Unimplemented
6	IBE	Bus Error on Instruction Fetch	12	Ov	Arithmetic Overflow Exception
7	DBE	Bus Error on Load or Store	13	Tr	Trap
8	Sys	Syscall Exception	15	FPE	Floating Point Exception

SIZE PREFIXES (10^x for Disk, Communication; 2^x for Memory)

SIZE	PREFIX	SIZE	PREFIX	SIZE	PREFIX	SIZE	PREFIX
10^3, 2^{10}	Kilo-	10^{15}, 2^{50}	Peta-	10^{-3}	milli-	10^{-15}	femto-
10^6, 2^{20}	Mega-	10^{18}, 2^{60}	Exa-	10^{-6}	micro-	10^{-18}	atto-
10^9, 2^{30}	Giga-	10^{21}, 2^{70}	Zetta-	10^{-9}	nano-	10^{-21}	zepto-
10^{12}, 2^{40}	Tera-	10^{24}, 2^{80}	Yotta-	10^{-12}	pico-	10^{-24}	yocto-

The symbol for each prefix is just its first letter, except μ is used for micro.

This question paper consists of 5 questions on 9 printed pages.

UCCD1133 INTRODUCTION TO COMPUTER ORGANISATION AND ARCHITECTURE

Appendix B: MIPS Instruction Syntax

Instruction	Syntax	Operation
add	`add rd, rs, rt`	[rd] = [rs] + [rt]
add immediate	`addi rt, rs, imm`	[rt] = [rs] + SignImm
add immediate unsigned	`addiu rt, rs, imm`	[rt] = [rs] + SignImm
add unsigned	`addu rd, rs, rt`	[rd] = [rs] + [rt]
and	`and rd, rs, rt`	[rd] = [rs] & [rt]
and immediate	`andi rt, rs, imm`	[rt] = [rs] & ZeroImm
branch if equal	`beq rs, rt, label`	if ([rs] == [rt]) PC = BTA
branch if not equal	`bne rs, rt, label`	if ([rs] != [rt]) PC = BTA
divide	`div rs, rt`	[lo] = [rs]/[rt],[hi] = [rs]%[rt]
divide unsigned	`divu rs, rt`	[lo] = [rs]/[rt],[hi] = [rs]%[rt]
jump	`j label`	PC = JTA
jump and link	`jal label`	$ra = PC + 4, PC = JTA
jump register	`jr rs`	PC = [rs]
load byte	`lb rt, imm(rs)`	[rt] = SignExt ([Address]7:0)
load byte unsigned	`lbu rt, imm(rs)`	[rt] = ZeroExt ([Address]7:0)
load upper immediate	`lui rt, imm`	[rt] = {imm, 16'b0}
load word	`lw rt, imm(rs)`	[rt] = [Address]
move from hi	`mfhi rd`	[rd] = [hi]
move from lo	`mflo rd`	[rd] = [lo]
move from/to coprocessor 0	`mfc0 rt, rd /mtc0 rt, rd`	[rt] = [rd]/[rd] = [rt]
move to hi	`mthi rs`	[hi] = [rs]
move to lo	`mtlo rs`	[lo] = [rs]
multiply	`mult rs, rt`	{[hi], [lo]} = [rs] × [rt]
multiply (32-bit result)	`mul rd, rs, rt`	[rd] = [rs] x [rt]
multiply unsigned	`multu rs, rt`	{[hi], [lo]} = [rs] × [rt]
nor	`nor rd, rs, rt`	[rd] = ~([rs] | [rt])
or	`or rd, rs, rt`	[rd] = [rs] | [rt]
or immediate	`ori rt, rs, imm`	[rt] = [rs] | ZeroImm
set less than	`slt rd, rs, rt`	[rs] < [rt] ? [rd] = 1 : [rd] = 0
set less than immediate	`slti rt, rs, imm`	[rs] < SignImm ? [rt] = 1 : [rt] = 0
set less than immediate unsigned	`sltiu rt, rs, imm`	[rs] < SignImm ? [rt] = 1 : [rt] = 0
set less than unsigned	`sltu rd, rs, rt`	[rs] < [rt] ? [rd] = 1 : [rd] = 0
shift left logical	`sll rd, rt, shamt`	[rd] = [rt] << shamt
shift right arithmetic	`sra rd, rt, shamt`	[rd] = [rt] >>> shamt
shift right logical	`srl rd, rt, shamt`	[rd] = [rt] >> shamt
store byte	`sb rt, imm(rs)`	[Address]7:0 = [rt]7:0
store word	`sw rt, imm(rs)`	[Address] = [rt]
subtract	`sub rd, rs, rt`	[rd] = [rs] – [rt]
subtract unsigned	`subu rd, rs, rt`	[rd] = [rs] – [rt]
xor	`xor rd, rs, rt`	[rd] = [rs] ^ [rt]
xor immediate	`xori rt, rs, imm`	[rt] = [rs] ^ ZeroImm